

Relatório Técnico — AgroSearch

UNIPÊ — Tendências em Ciência da Computação | Alunas: Anna Maria, Maria Eduarda e Milena Azevêdo

1. Arquitetura da Solução

O AgroSearch é um protótipo de motor de busca textual para a base de manuais técnicos da AgroTech Solutions (agricultura sustentável, controle de pragas e irrigação), implementado em Python/Streamlit em um único arquivo (`app_agrosearch.py`). A base é carregada de forma estática em um dicionário com 5 documentos. O pipeline segue três fases: (1) pré-processamento do texto, (2) construção do índice invertido e (3) ranqueamento por TF-IDF, todas exibidas em uma interface interativa. Seguindo a restrição do desafio, o pré-processamento, o índice e o TF-IDF são implementados do zero, sem `scikit-learn`, `TfidfVectorizer` ou bibliotecas de PLN como o `NLTK`: usamos apenas a biblioteca padrão do Python (`re`, `unicodedata`, `collections`) e o `Streamlit` para a interface.

2. Pré-processamento (Fase 1)

O pipeline aplica quatro etapas em sequência e é exposto pela função `preprocessar(texto, usar_stopwords, usar_stemming)`, reutilizada nos documentos e na consulta do usuário, de modo que ambos recebam exatamente o mesmo tratamento. **(1) Tokenização:** a expressão regular `[^\W_]+` extrai sequências de letras e dígitos, descartando a pontuação e mantendo as palavras acentuadas inteiras. **(2) Normalização:** conversão para minúsculas e remoção de acentos via decomposição Unicode (NFKD), de modo que “Irrigação” e “irrigacao” viram o mesmo token. **(3) Stopwords:** lista própria com 130 palavras do português (artigos, preposições, pronomes, conjunções e verbos auxiliares como “ser” e “pode”), normalizada com a mesma função dos tokens para que “é”, “até” e “não” sejam reconhecidas depois da remoção dos acentos. **(4) Stemming:** redutor de sufixos para o português escrito do zero, inspirado no RSLP, em três passos: redução de plural (lagartas → lagarta; animais → animal), remoção do maior sufixo que case com a tabela de regras, respeitando um tamanho mínimo de radical (irrigação → irrig; desfolhadoras → desfolh; leguminosas → legumin), e remoção da vogal final após consoante (controle → control).

Na interface, dois checkboxes (**Remover stopwords** e **Aplicar stemming**) ligam e desligam essas etapas e o vocabulário é recalculado a cada alteração. A tela também traz o passo a passo de cada documento, o vocabulário com a frequência de cada termo, um quadro com as palavras agrupadas em cada radical e um campo de teste para consultas livres, útil para conferir como a query será tratada na Fase 3.

Token original	Após normalização	Stopword?	Resultado final (com stemming)
A	a	sim	removida
irrigação	irrigacao	não	irrig
floração	floracao	não	flor
leguminosas	leguminosas	não	legumin
desfolhadoras	desfolhadoras	não	desfolh
para	para	sim	removida
algodão	algodao	não	algodao
Trichogramma	trichogramma	não	trichogramm

Tabela 1 – Exemplos do pipeline sobre palavras dos documentos (stopwords e stemming ligados).

Configuração dos checkboxes	Termos no vocabulário	Total de tokens
Sem stopwords, sem stemming (só normalização)	47	66
Com remoção de stopwords	33	37
Com stemming	47	66
Com stopwords e stemming (padrão)	33	37

Tabela 2 – Tamanho do vocabulário dos 5 documentos em cada configuração.

Resultados. A remoção de stopwords reduz o vocabulário de 47 para 33 termos (cerca de 30%) e o total de tokens de 66 para 37 (cerca de 44%): quase metade do texto eram palavras sem valor de busca. O stemming não altera o tamanho do vocabulário neste corpus pequeno, porque nenhuma palavra distinta compartilha radical com outra dentro dos documentos, mas muda a forma dos termos e é decisivo na consulta: “irrigação da soja” e “irrigar soja” geram os mesmos tokens (`irrig`, `soj`), que casam com os Docs 1 e 5 (`irrig`) e com os Docs 1, 2 e 4 (`soj`). **Limitação.** Por ser heurístico e simplificado, o stemmer gera radicais que não são palavras (`soja` → `soj`) e não agrupa algumas formas relacionadas, como `cultivo` (`cultiv`) e `cultura` (`cultur`).

3. Índice Invertido (Fase 2)

Estrutura. O índice invertido é um dicionário em memória, construído do zero com `dict` e `set`, no formato Termo → [IDs de Docs]. Cada documento passa pela função `preprocessar` da Fase 1 e cada token final é registrado com o ID do documento, sem repetição. Assim, o índice contém apenas tokens limpos (minúsculas, sem acentos, sem stopwords e com stemming) e “irrigação” e “irrigar” caem no mesmo termo. Uma versão estendida (`construir_indice_com_frequencias`) guarda também a contagem por documento, base do TF na Fase 3.

Exibição. O índice é mostrado em `st.dataframe` ou `st.json`, com filtro por termo e consulta rápida OR/AND, e é reconstruído a cada mudança dos checkboxes (33 termos com stopwords e stemming; 47 sem eles). **Exemplo.** `irrig` → [1, 5]; `soj` → [1, 2, 4]; a consulta “irrigação da soja” com AND retorna apenas o Doc 1.

4. Busca e Ranqueamento TF-IDF (Fase 3)

Fórmulas. $TF(t,d) = f(t,d) / |d|$ (ocorrências do termo divididas pelo nº de tokens do documento, para não favorecer textos longos); $IDF(t) = \log_{10}(N / df(t))$, com $N = 5$ documentos e $df =$ nº de documentos que contêm o termo; $TF-IDF(t,d) = TF \cdot IDF$. O **score** de cada documento é o TF-IDF acumulado, isto é, a soma dos pesos dos termos distintos da consulta. **Implementação.** Tudo foi feito do zero com `math` e `collections`: o TF sai direto das contagens do índice com frequências da Fase 2 (`construir_indice_com_frequencias`), e a consulta passa pela mesma função `preprocessar` dos documentos, com a configuração dos checkboxes da Fase 1. Termos ausentes do corpus são ignorados e avisados na tela, e empates no topo são destacados juntos. A aba Fase 3 mostra o IDF de cada termo da consulta, a tabela ordenada do maior para o menor TF-IDF acumulado com o vencedor em verde e, num painel expansível, as matrizes TF, IDF e TF-IDF de todo o vocabulário.

Posição	Documento	TF-IDF(irrig)	TF-IDF(soj)	TF-IDF acumulado	Cosseno
1º	Doc 1 (soja + irrigação)	0,0497	0,0277	0,0775	0,2572
2º	Doc 5 (gotejamento)	0,0568	0,0000	0,0568	0,1977
3º	Doc 2 (controle biológico)	0,0000	0,0317	0,0317	0,0664
4º	Doc 4 (lagartas)	0,0000	0,0277	0,0277	0,0610
5º	Doc 3 (adubação verde)	0,0000	0,0000	0,0000	0,0000

Tabela 3 – Ranking da consulta “irrigação da soja” (tokens: `irrig`, `soj`; stopwords e stemming ligados).

Teste. O Doc 1 vence por ser o único com os dois termos. O Doc 5 fica em 2º apesar de ter só “irrig”, porque esse termo é mais raro que “soj” ($IDF\ 0,398 \times 0,222$) e o Doc 5 é mais curto (7×8 tokens). “Irrigar soja” gera os mesmos tokens e o mesmo ranking, graças ao stemming. Com as **stopwords desligadas**, porém, o vencedor passa a ser o Doc 4: o “da” da consulta só aparece nesse documento, recebe o IDF máximo (0,699) e supera os termos de conteúdo, o que mostra na prática por que a Fase 1 é necessária. **Bônus (cosseno).** $\cos(q,d) = q \cdot d / (||q|| ||d||)$, com a consulta e os documentos representados como vetores TF-IDF sobre todo o vocabulário. Ao contrário da soma, o cosseno normaliza pelo vetor inteiro do documento, penalizando textos com muitos termos fora da consulta. Nas consultas testadas, os dois critérios produziram a mesma ordem, e o app mostra os dois rankings lado a lado.

5. Divisão de Tarefas

Aluna	Responsabilidade
Maria Eduarda	Fase 1 (Pipeline de pré-processamento: tokenização, normalização, stopwords e stemming) e checkboxes de Stemming e Stopwords na interface
Milena Azevêdo	Fase 2 (Índice invertido Termo → [IDs de Docs], versão com frequências por documento e exibição via <code>st.dataframe/st.json</code> com consulta OR/AND)
Anna Maria	Fase 3 (TF, IDF e TF-IDF do zero, ranqueamento com destaque do documento vencedor e bônus de similaridade de cosseno), integração final do <code>app_agrosearch.py</code> e relatório

6. Conclusão

As três fases funcionam integradas em um único app (`app_agrosearch.py`), uma aba por fase, todas lendo a mesma configuração de pré-processamento. Mudar um checkbox na Fase 1 altera o vocabulário, o índice invertido e o ranking ao mesmo tempo. O principal aprendizado foi ver a dependência entre as etapas: a qualidade do TF-IDF depende diretamente do pré-processamento, já que sem stopwords uma preposição rara chegou a decidir a busca. Com um corpus de apenas 5 documentos, o IDF é muito sensível: um termo presente em todos os documentos zera, e um termo exclusivo recebe peso máximo. Uma base maior e o uso do cosseno como critério principal tornariam o ranqueamento mais estável.